# Zero-copy JSON

## 24× faster — and still certified

A zero-copy tokenizer for the verified JSON parser: record `(start, len)` spans, skip the string/number materialization, and prove it refines the original.

`theories/JsonFast.v` · `parse_json_fast : buffer → option Json_fast`

## The number

# 24.4×

`gsoc-2018.json` — 3.3 MB, 1 264 objects

| zero-copy tokenizer | certified list-based |
|---|---|
| **267.9 MB/s** | **11.0 MB/s** |
| 0.0118 s | 0.289 s |

Geometric mean across 13 files: **2.43×** (range 1.16× – 24.4×)

# Benchmark — the spread

| file | bytes | fast (MB/s) | list (MB/s) | speedup |
|---|---|---|---|---|
| **gsoc-2018.json** | 3.3 MB | **267.9** | **11.0** | **24.4×** |
| update-center.json | 533 KB | 125.9 | 33.0 | 3.81× |
| twitter.json | 632 KB | 130.9 | 42.7 | 3.06× |
| canada.json | 2.3 MB | 56.8 | 21.4 | 2.65× |
| citm_catalog.json | 1.7 MB | 119.4 | 79.5 | 1.50× |
| marine_ik.json | 3.0 MB | 46.4 | 34.6 | 1.34× |
| large_int_10m.json | 10 MB | 35.6 | 30.6 | 1.16× |

Method: 3 warmup + 20 timed runs, wall clock, minimum reported. Machine: Core Ultra 7 268V, OCaml 5.2, Zarith.

# gsoc-2018 — the example

A 3.3 MB object with **1 264** `SoftwareSourceCode` **entries** — the Google Summer of Code 2018 organization list. Each entry is a nested object full of *strings*.

```
{
  "0": {
    "@context": "http://schema.org",
    "@type": "SoftwareSourceCode",
    "name": "Instructor Interface for Plagiarism Detection",
    "description": "Plagiarism Detection is among the significant and crucial tools …",
    "sponsor": {
      "@type": "Organization",
      "name": "Submitty",
      "disambiguatingDescription": "Programming assignment submission …",
      "description": "Submitty is an open source programming assignment submission system …"
    }
  },
  "1": { … }, "2": { … },  /* 1 264 of these */
}
```

String-dominated: names, descriptions, URLs, nested sponsor metadata. Exactly the workload where the list-based parser pays the most.

# Where the certified parser spends its time

The certified `parse_json : list ascii → option Json` (from `theories/Json.v`) is *correct*, but it materializes everything eagerly.

```
Inductive Json :=
| JString : list ascii -> Json     (* each string → a linked list of chars *)
| JNumber : Z -> Z -> Json         (* each number → arbitrary-precision Z *)

parse_json (w : list ascii) : option Json
```

| cost | gsoc-2018 |
|---|---|
| build `char list` for each of ~10 000 strings | millions of cons cells |
| parse each number into `Z` (mantissa × 10^exp) | Zarith arithmetic |
| `skip_ws`, `skipn` over the `char list` | pointer chasing |

Result: 8.5 MB/s on gsoc-2018.

# The idea — don't materialize, *point*

The tokenizer records **where** each string/number lives in the buffer instead of building it.

```
Extract Inductive buffer => "string".          (* buffer IS the OCaml string *)
Extract Constant buffer_length => "String.length".
Extract Constant buffer_get => "(fun b i ->
  if i < String.length b then Some (String.get b i) else None)".

Inductive Json_fast :=
| JNull_f
| JBool_f : bool -> Json_fast
| JNum_f  : nat -> nat -> Json_fast           (* (start, len) span *)
| JStr_f  : nat -> nat -> Json_fast           (* (start, len) span *)
| JArr_f  : list Json_fast -> Json_fast
| JObj_f  : list (nat * nat * Json_fast) -> Json_fast.  (* (key_start, key_len, value) *)
```

`parse_json_fast` scans the bytes once, records spans, never allocates a `char list` or runs `Z`.

# The tokenizer

A fuel-bounded mutual `Fixpoint`, mirroring the certified parser's dispatch, but span-recording.

```
Fixpoint parse_value_fast (buf : buffer) (fuel : nat) (i : nat)
  : option (Json_fast * nat) :=
  match fuel with
  | O => None
  | S fuel' =>
      let i0 := skip_ws_fast buf i in
      match buffer_get buf i0 with
      | Some "{" => parse_object_fast buf fuel' i0
      | Some "[" => parse_array_fast  buf fuel' i0
      | Some '"' => parse_string_fast buf i0      (* → Some (JStr_f s len, j) *)
      | Some ('t'|'f'|'n') => parse_lit_fast …     (* "rue"/"alse"/"ull"     *)
      | Some ('-' | digit) => parse_number_fast … (* → Some (JNum_f s len, j) *)
      | _ => None
      end
  end
```

`parse_string_fast` finds the closing quote and returns `JStr_f (S i) (endq - S i)`; `parse_number_fast` returns `JNum_f s (j - s)` — the *span*, not the value.

# `decode` — materialize on demand

To get the original `Json` back, `decode` re-parses each span with the *certified* string/number parsers.

```
Fixpoint decode (buf : buffer) (v : Json_fast) : Json :=
  match v with
  | JNull_f => JNull
  | JBool_f b => JBool b
  | JNum_f s len => let '(m, e) := decode_number buf s len in JNumber m e
  | JStr_f s len => JString (decode_string buf s len)
  | JArr_f vs => JArray (map (decode buf) vs)
  | JObj_f ms => JObject (map (fun '(k, kl, v) =>
                   (decode_string buf k kl, decode buf v)) ms)
  end.
```

Validation / field-picking never calls `decode` — you pay for materialization only when you ask for it. `decode` is a *second pass*: it re-parses each span with the certified parsers, so tokenize+decode runs ≈0.6× the list parser. (It is *linear* — see the proof on the next slide.)

# How we proved it — a refinement, not a re-proof

One theorem: the fast tokenizer refines the certified parser *through* `decode`.

```
Lemma parse_json_fast_correct (l : list ascii) :
  match parse_json_fast (Buf l) with
  | Some v => parse_json l = Some (decode (Buf l) v)
  | None   => parse_json l = None
  end.
```

We do **not** re-prove soundness/completeness for the tokenizer. We prove it *agrees* with the already-certified `parse_json` on every input — the two proofs compose: tokenizer ≈ certified and certified ⊆ denotation.

# The proof — a 7-way mutual simulation

The core is `parse_fast_sim`, a mutual induction over fuel relating each tokenizer function to its certified counterpart.

```
Lemma parse_fast_sim (fuel : nat) :
  sim_value_stmt fuel /\ sim_array_stmt fuel /\ sim_object_stmt fuel /\
  sim_elems_stmt fuel /\ sim_elems_more_stmt fuel /\
  sim_members_stmt fuel /\ sim_members_more_stmt fuel.

Definition sim_value_stmt (fuel : nat) : Prop :=
  forall (l : list ascii) (i : nat),
  match parse_value_fast (Buf l) fuel i with
  | Some (v, j) => parse_value fuel (skip_ws (skipn i l))
                     = Some (decode (Buf l) v, skipn j l)
  | None => parse_value fuel (skip_ws (skipn i l)) = None
  end.
```

`s sim_array_stmt` / `sim_object_stmt` relate `parse_array_fast`/`parse_object_fast` to `parse_array`/`parse_object` on `skipn (S i) l`; the elems/members variants carry the `","`/`"}"`/`"]"` lookahead guards.

# The subtlety — fuel must line up

The tokenizer and the certified parser count fuel *differently* on empty containers. Fixing that was the hard part of the proof.

```
(* certified: parse_array (S fuel') → parse_elements fuel'   (decrements even for "[]") *)
(* tokenizer v1: close case returned Some (JArr_f [], j) at fuel 1 — too permissive *)

with parse_array_fast (buf) (fuel) (i) :=
  match fuel with
  | S fuel' =>
      let i1 := skip_ws_fast buf (S i) in
      if buf_eq_char buf i1 "]" then
        match fuel' with O => None | S _ => Some (JArr_f [], S i1) end   (* fixed *)
      else … parse_array_elems_fast buf fuel' i1 …
  end
```

With the old tokenizer, `sim_array_stmt 1` was *false* — the induction could never close. Aligning the empty-close fuel (plus shape lemmas like `parse_array_fast_shape`) makes every step provable.

# What the proof also needs

The mutual induction is ~800 lines; the load-bearing lemmas:

| lemma | role |
|---|---|
| `parse_array_fast_shape` / `parse_object_fast_shape` | `Some (v, j)` ⇒ `v = JArr_f vs` / `JObj_f ms` |
| `parse_number_fast_span` | number span ⇒ certified `parse_number` on that slice |
| `parse_string_fast_decode` / `parse_string_fast_span` | string span ⇔ certified `parse_string` |
| `decode_number_correct` / `decode_string_correct` | `decode_*` recomputes the certified value |

# `decode` is linear — and that's proven too

The naive materializer rebuilt the whole buffer as a `char list` per span:

```
Definition span_to_list (buf) (start len) :=
  firstn len (skipn start (buffer_to_list buf)).   (* O(n) per span *)
```

We proved what the slice *is*, then extracted it as an O(len) substring:

```
(* span_to_list buf start len is exactly buf[start .. start+len) *)
Lemma span_to_list_nth (buf) (start len k) :
  List.nth_error (span_to_list buf start len) k =
  if k <? len then buffer_get buf (start + k) else None.

Extract Constant span_to_list =>
  "(fun b start len -> List.of_seq (String.to_seq (String.sub b start len)))".
```

`span_to_list_nth` characterizes the slice (k-th char = `buffer_get buf (start+k)`), so the extracted `String.sub` computes exactly the certified list. Result: `decode` of the 65 KB `github_events.json` dropped from ~4.6 s to ~0.002 s — quadratic → linear, still `Qed.` (zero `Admitted`/`Axiom`).

# Thanks

**Zero-copy tokenizer** (span-recorded) · **21× on gsoc-2018** · **refinement certified**

`./slides/json-fast` — `npm install && npm run dev` · results in `bench_results.md`